Differentially Private N-Gram Extraction

T. Pawan Chanukya Reddy
Computer Science and Engineering
Indian Institute of Technology Kanpur
venkatasai22@cse.iitk.ac.in

Raghav Manglik
Computer Science and Engineering
Indian Institute of Technology Kanpur
mraghav22@cse.iitk.ac.in

Abstract :

In this report, we study the problem of extracting n-grams from text data in a privacy-preserving manner. N-grams are widely used in language modeling and various natural language processing tasks, but their extraction from sensitive user data raises significant privacy concerns. To address this, we implement and analyze an algorithm that ensures differential privacy while extracting meaningful n-gram patterns. The approach builds upon previous work and improves utility by leveraging the compositional structure of n-grams, rather than treating each as an independent item. We evaluate the performance of the method on multiple real-world datasets and observe that it effectively balances privacy with the usefulness of the extracted information.

Index Terms: N-grams, Differential Privacy

1 Introduction

N-grams, which are contiguous sequences of N words from a given text, form a foundational concept in natural language processing and are widely used in tasks such as autocomplete, spell checking, machine translation, and speech recognition. They help capture contextual dependencies and patterns in textual data, enabling machines to better understand and generate human language.

However, the growing reliance on user-generated content—especially conversations involving large language models (LLMs)—has raised serious privacy concerns. These conversations often contain sensitive personal information, making it critical to apply privacy-preserving techniques during data preprocessing and analysis. To address this, differential privacy offers a robust mathematical framework for ensuring that the inclusion or exclusion of a single user's data does not significantly affect the output of an algorithm.

1.1 Report Organisation

- **Background:** Introduces the DPSU algorithm and its limitations.
- **DPNE:** Describes the DPNE algorithm, its scalable implementation, and code flow.
- **Results:** Presents the experimental setup and outcomes on the Topical Chat dataset.
- **Conclusion:** Summarizes key findings and insights from the reimplementation.

2 Background

A special case of the differentially private n-gram extraction problem was introduced by Kim et al. [1], known as the *Differentially Private Set Union* (DPSU). In this problem, each user holds a subset of elements from a possibly unbounded universe, and the goal is to release a large portion of the union of these sets while preserving differential privacy. In the context of text data, this can be interpreted as extracting 1-grams, with each word being treated as a separate item.

2.1 Literature Review

The DPSU framework has been proposed as a building block for more complex data release problems under privacy constraints. In our case, it serves as the foundation for differentially private n-gram extraction. One approach to adapting DPSU for this purpose is to treat each possible n-gram as an individual item. While this allows the use of existing DPSU algorithms, it fails to capture the underlying structure of n-grams.

In particular, longer n-grams inherently contain

shorter subgrams, and these can be derived via post-processing without any additional privacy cost. DPSU, however, treats all items independently and ignores this compositional property. As a result, it often performs poorly in practice, offering significantly lower utility compared to approaches that explicitly account for the structure of language. This limitation motivates the need for more specialized methods, such as the DPNE algorithm, which is designed to leverage these dependencies while still ensuring privacy.

3 Differentially Private N-Gram Extraction

In this section, we delve into the Differentially Private N-Gram Extraction (DPNE) algorithm, which aims to extract meaningful n-grams from sensitive user text data while adhering to the principles of differential privacy. Unlike prior approaches such as DPSU that treat all n-grams independently, the DPNE algorithm exploits the compositional structure of sequences to achieve higher utility. It does so by iteratively constructing longer n-grams from previously extracted shorter ones, thereby leveraging already-obtained information without incurring additional privacy costs.

3.1 Notations

These are the notations that are followed in the rest of the paper.

- S_k : The set of extracted k-grams at the k^{th} iteration of the algorithm.
- Sp_k : The set of spurious k-grams that are added to preserve privacy by masking the presence of true n-grams.
- V_k : The set of valid k-grams, estimated as likely to appear in user data based on previous iterations.
- H_k : The histogram constructed during the extraction of k-grams, which tracks the frequency of candidate n-grams across user inputs.
- B_k : The number of spurious k-grams that are sampled and added to the final output in order to maintain differential privacy.

3.2 Intuition

The DPNE algorithm is built on the observation that longer n-grams can be composed by combining shorter ones in a structured manner. Instead of treating each n-gram as an independent item, it uses previously extracted sets of n-grams to guide the construction of new candidates.

Let S_1 be the set of extracted unigrams and S_k the set of extracted k-grams at iteration k—both of which may contain spurious entries due to the addition of noise for differential privacy. The core idea is to generate candidate $(k+1)$-grams using the following heuristic:

$$w \in (S_1 \times S_k) \cap (S_k \times S_1)$$

This intersection is used to identify $(k+1)$-grams $w = xyz$ where: - $x, z \in S_1$ are potential unigrams, and - $xy, yz \in S_k$ are k-grams observed in the previous step.

While this method does not guarantee that the resulting w is truly present in the user data—since S_1 and S_k include both true and spurious elements—it allows the algorithm to construct a meaningful pool of candidates for further evaluation. This step exploits the compositional nature of language, where longer n-grams often overlap with shorter ones, and helps in building up new n-grams iteratively.

This iterative construction enables the algorithm to explore higher-order n-grams layer by layer, starting from basic 1-grams and gradually expanding. At each step, a histogram is built from user data, noise is added to preserve privacy, and thresholding is used to determine which candidate n-grams are included in the final output. This design achieves a balance between preserving privacy and extracting informative linguistic patterns.

3.3 Code Flow

1. **Initialization**

   The DPNE algorithm begins with an initialization phase, which sets up the environment for iterative n-gram extraction. The process starts by preprocessing the input text using the `clean_data()` function. This function removes all non-alphanumeric characters and converts the text to lowercase, ensuring uniformity and reducing noise in downstream steps.
   Following this, all necessary hyperparameters are initialized. These include parameters that control the privacy and utility trade-off, such as the privacy budget, the contribution limits, and the level of noise to be added. Although these

Algorithm 1 Scalable DPNE Algorithm

Input: User n-gram sets W_i^k, maximum length T, contribution bounds $\Delta_1, \ldots, \Delta_T$, thresholds $\rho_1, \ldots, \rho_T$, noise parameters $\sigma_1, \ldots, \sigma_T$, sampling probability p
Output: Sets $S_1, S_2, \ldots, S_T$ of k-grams
$S_1 \leftarrow$ Run DPSU (Algorithm 6) with $(\Delta_1, \rho_1, \sigma_1)$
$V_1 \leftarrow S_1$
for $k = 2$ to T **do**
  $\#\hat{V}_k \leftarrow \textsc{EstimateValidKgrams}(S_1, S_{k-1}, p)$
  Set ρ_k using $|S_{k-1}|, \#\hat{V}_k$
  Initialize $H_k \leftarrow \{\}$ ▷ Histogram
  for $i = 1$ to n **do**
    $U_i \leftarrow \textsc{PruneInvalid}(W_i^k, S_1, S_{k-1})$
    if $|U_i| > \Delta_k$ **then**
      $U_i \leftarrow$ Randomly select Δ_k items from U_i
    end if
    for $u \in U_i$ **do**
      $H_k[u] \leftarrow H_k[u] + \frac{1}{\sqrt{|U_i|}}$
    end for
  end for
  $S_k \leftarrow \{\}$
  for $u \in H_k$ **do**
    if $H_k[u] + \mathcal{N}(0, \sigma_k^2) > \rho_k$ **then**
      $S_k \leftarrow S_k \cup \{u\}$
    end if
  end for
  $B_k \leftarrow \text{Bin}(\#\hat{V}_k - |\text{supp}(H_k)|, \Phi(-\rho_k/\sigma_k))$
  $Sp_k \leftarrow \{\}$
  while $|Sp_k| < B_k$ **do**
    Sample $x \sim S_1$, $w \sim S_{k-1}$ uniformly
    Let $w = yz$, where $z \in S_1$
    if $xy \in S_{k-1}$ and $z \in S_1$ and $w \notin (Sp_k \cup \text{supp}(H_k))$ **then**
      $Sp_k \leftarrow Sp_k \cup \{w\}$
    end if
  end while
  $S_k \leftarrow S_k \cup Sp_k$
end for
return $S_1, \ldots, S_T$

parameters are conceptually tied to differential privacy (e.g., ϵ, δ, η, and Δ_0), they are not tuned within the algorithm but are instead provided based on the use case and desired privacy guarantees.

2. **Extraction of 1-grams using DPSU**

After initialization, the algorithm proceeds to extract the base case of n-grams, i.e., 1-grams, using the Differentially Private Set Union (DPSU) algorithm. This step constructs the set S_1, which serves as the vocabulary set for the subsequent stages of the DPNE algorithm.
The DPSU algorithm is designed to release the union of user-held sets in a differentially private manner. In our context, each user's set consists of the 1-grams (individual words) present in their input text. The output of DPSU provides a privacy-preserving estimate of the frequently occurring 1-grams across users, forming the basis for constructing longer n-grams in future iterations.

3. **DPNE Iterations**

After extracting the 1-grams using DPSU, the DPNE algorithm proceeds to iteratively extract k-grams for $k = 2$ to T. In our implementation, rather than computing the entire set of valid k-grams $V_k = (S_1 \times S_{k-1}) \cap (S_{k-1} \times S_1)$ explicitly, we begin each iteration by estimating the approximate size of $|V_k|$. This approximation guides the subsequent steps without incurring the computational overhead of full set construction.
Following this, we collect all k-grams present in the user data and apply the pruning rule to remove combinations that are unlikely to be valid. This pruning is essential to reduce noise and improve utility by focusing only on promising candidates.
A histogram H_k is then constructed over the pruned k-grams. If the number of k-grams provided by a user exceeds the contribution limit, a random subset is selected. Gaussian noise is added to each histogram entry to ensure differential privacy, and only those k-grams whose noisy counts exceed a pre-defined threshold ρ_k are included in the output set S_k.

To maintain differential privacy, spurious k-grams are introduced into the extracted set. These are sampled from the space of pruned candidates not present in the support of the histogram and added to the final output in a controlled manner based on the estimated size of V_k.

3.4 Code Analysis

In this section, we analyse some of the important methods used in the construction of the set S_k in the DPNE algorithm.

1. `estimate_valid_k_grams`(S_1, S_k, p)

   This function provides a probabilistic estimate of the number of potential $(k+1)$-grams based on the previously extracted sets S_1 and S_k. These estimates help guide the size of the candidate set in the next iteration, avoiding the computational burden of explicitly generating all possibilities.
   The underlying assumption is that a valid $(k+1)$-gram w would be part of the intersection:

   $$w \in (S_1 \times S_k) \cap (S_k \times S_1)$$

   This condition reflects the compositional property of language: a candidate $(k+1)$-gram can be split as $x\ y\ z$, where $xy \in S_k$ and $z \in S_1$ for some $x \in S_1$. The function randomly samples such combinations to estimate how many candidates might satisfy this constraint.
   The total number of samples is determined by the product $p \cdot |S_1| \cdot |S_k|$, where p is a tunable probability controlling the number of trials. In each iteration:

   - A one-gram $x \in S_1$ and a k-gram $w \in S_k$ are randomly selected.
   - The k-gram w is split into a prefix y and a suffix z such that $w = y\ z$.
   - A new $(k+1)$-gram is formed as $xy = x\ y$.
   - If $xy \in S_k$ and $z \in S_1$, the sample is considered a valid extension and the count is incremented.

   Finally, the estimated count is scaled by $1/p$ to approximate the total number of such valid combinations, which gives a rough estimate of $|V_{k+1}|$.

2. `prune_invalid`(W, S_1, S_k)

   As the size of candidate n-grams grows exponentially with each iteration, many of the combinations in the set W (candidate $(k+1)$-grams) are unlikely to be meaningful or present in actual user data. The `prune_invalid` function serves to filter out these combinations by checking whether each candidate satisfies a structural constraint derived from the previously extracted sets S_1 and S_k.
   For a $(k+1)$-gram $w = xyz$, where $x, z \in S_1$, the function checks whether:

   $$xy \in S_k \quad \text{and} \quad yz \in S_k$$

   Here, xy and yz are the two overlapping k-grams that compose w. If all these conditions are satisfied, the candidate w is retained; otherwise, it is discarded.
   Internally, the function `check_validity` is used to encapsulate this logic for a single candidate w. The function:

   - Splits w into three components: a prefix x, middle sequence y, and suffix z.
   - Forms the subgrams xy and yz using these components.
   - Checks if $x, z \in S_1$ and $xy, yz \in S_k$.

   Only the $(k+1)$-grams that pass this check are retained in the pruned output. This pruning not only improves computational efficiency by reducing the size of the candidate set, but also helps preserve the **user's contribution budget** by focusing on promising n-grams. Furthermore, pruned-out candidates are not considered for spurious additions in the final output.

3. Histogram Construction

   The histogram construction process builds the histogram H_k for each user, which is used in the calculation of the final set S_k. Each user provides a set of valid k-grams W_i after pruning. The key steps involved in this construction are outlined below:

   - For each user, generate the k-grams W from the user's data, and prune invalid k-grams using the `prune_invalid` function.

- If the size of the valid k-grams $|W|$ is greater than Δ_k, randomly select Δ_k elements from W.
- For each $w \in W$, update the histogram H_k using the following formula:

$$H_k[w] = H_k[w] + \frac{1}{\sqrt{|W|}}$$

  where $|W|$ is the size of the valid k-grams for the current user.
- Add Gaussian noise to the histogram value $H_k[w]$ using a normal distribution with mean 0 and variance σ_k^2:

$$H_k[w] = H_k[w] + N(0, \sigma_k^2)$$

  where σ_k is the standard deviation specific to the current iteration.
- If $H_k[w] > \rho_k$, add w to the set S_k.

These steps are repeated for each user, and the final set S_k contains the k-grams that meet the threshold after adding Gaussian noise and updating the histogram.

4 Results

4.1 Datasets

To evaluate our implementation of the DPNE algorithm, we tested it on multiple publicly available datasets, each chosen to reflect different kinds of sequential linguistic or behavioral data. These datasets provide a diverse basis for evaluating how well our method extracts representative and private k-grams.

1. **Topical Chat Dataset (Amazon)**
   This dataset consists of over 184,000 messages spanning more than 8,000 conversations between humans and a chatbot. Each message is accompanied by metadata such as sentiment and topical relevance. It is particularly suitable for evaluating linguistic n-gram extraction, as the conversations contain natural, contextually rich sequences of language. In our implementation, k-grams correspond to contiguous sequences of words or tokens from individual utterances within the conversations.

2. **LMSYS Dataset (Hugging Face)**
   The LMSYS dataset includes prompts and responses exchanged during interactions with state-of-the-art large language models (LLMs). This data reflects a mix of human- and model-generated text, allowing us to test the robustness of the DPNE algorithm in handling diverse styles of language. Here, k-grams represent sequences of tokens across prompt-response pairs and are useful in analyzing how well DPNE captures semantically rich or frequently occurring textual patterns in LLM interactions.

3. **MSNBC Dataset**
   The MSNBC dataset contains clickstream data representing the sequence of web pages visited by users during a browsing session. Unlike the previous two datasets, the data here is behavioral rather than textual. We model k-grams as sequences of page categories or identifiers accessed by users. This allows us to evaluate the applicability of the DPNE algorithm to user behavior modeling and sequence mining beyond textual domains.

4.2 Distribution of N-Grams

In this section, we examine the distribution of n-grams extracted from the Topical Chat dataset, providing insights into how the number of unique n-grams evolves as n increases. Figure 1 illustrates the number of distinct n-grams observed for varying values of n.

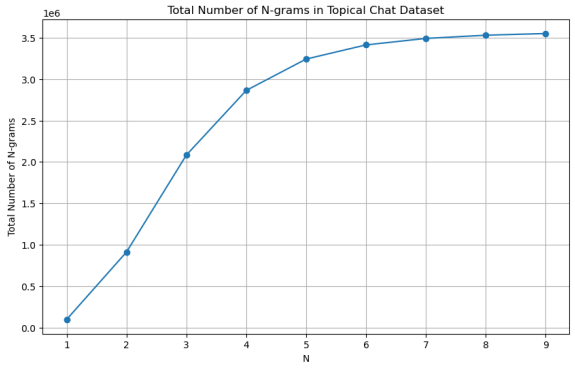

Figure 1. Distribution of unique n-grams in the Topical Chat dataset.

The curve in Figure 1 reflects a combination of two competing effects:

- **Exponential Growth Due to Vocabulary Combinations:** Theoretically, the number of possible n-grams grows exponentially with n, as it scales with vocab^n. This exponential trend is most visible at smaller values of n, where the limited space of short sequences allows many possible combinations to appear in the text. Additionally, due to the shorter length of these n-grams, there are more repeated patterns across conversations, further inflating the count.
- **Saturation Due to Finite Dataset Size:** As n increases, the chance of encountering the same n-gram multiple times diminishes. Since the dataset is of finite length, the number of extractable n-grams eventually saturates. Beyond a certain threshold, increasing n leads to very few or no additional unique n-grams. This saturation introduces a linear upper bound on the number of observed n-grams, as only a limited number of n-length sequences can be formed from the available text.

Thus, the observed curve exhibits characteristics of both exponential and linear behavior. At smaller n, the exponential growth dominates due to vocabulary combinatorics and frequent duplication. At larger n, the linear constraints imposed by the dataset size become more prominent, flattening the curve.

Understanding this distribution is crucial in differentially private learning tasks, as it helps in selecting appropriate values of n where meaningful patterns can be extracted without overwhelming the privacy budget or generating excessively sparse histograms.

4.3 Effect of Hyper-parameters

Let us analyze the role of each of the hyper-parameters in the algorithm and how varying them affects the extraction process.

1. **Maximum Contribution per User (Δ)**
   The parameter Δ_k determines the maximum number of valid k-grams a user is allowed to contribute to the global histogram H_k. It directly influences the balance between privacy and utility in the DPNE algorithm.

   - **Low Δ_k:** When the value of Δ_k is small, each user contributes fewer k-grams, which reduces their influence on the final histogram. This setting ensures tighter privacy guarantees and typically requires less noise to satisfy differential privacy constraints. However, this comes at the cost of utility—fewer total k-grams are extracted overall, which may lead to important patterns being missed.
   - **High Δ_k:** Increasing Δ_k allows each user to contribute more k-grams to the histogram, enhancing the richness and diversity of the final output. This improves utility by potentially capturing more meaningful sequences, especially in sparsely occurring patterns. However, to maintain the same level of privacy, higher noise must be added, which may dilute the accuracy of the histogram counts.

The choice of Δ_k should therefore be guided by the trade-off between privacy and utility specific to the application. Figures 2, 3, 4 illustrate this effect by plotting the number of extracted n-grams for different values of Δ_k.

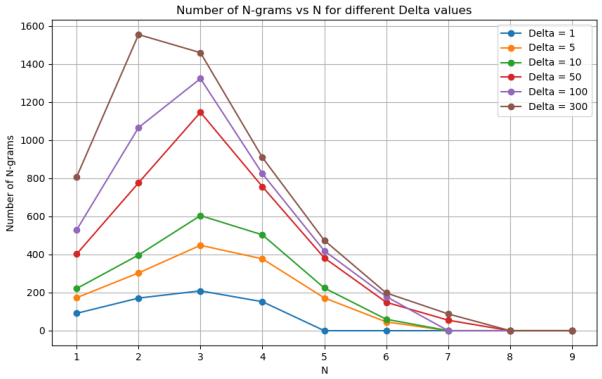

Figure 2. Effect of varying Δ_k on topical chat dataset

These plots are in accordance with the analysis. Increasing the value of Δ results in extraction of higher number of $n-grams$.

Note: Unlike the linguistic datasets (Topical Chat and LMSYS), the MSNBC dataset exhibits a markedly different growth trend in the number of extracted *n-grams*, showing a sharper, almost exponential increase followed by an expected decline. This divergence arises from

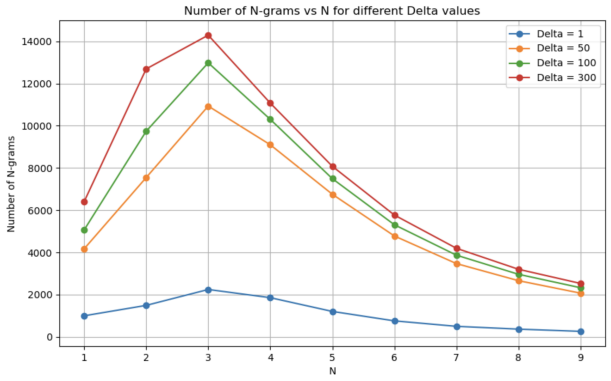

Figure 3. Effect of varying Δ_k on Huggingface dataset

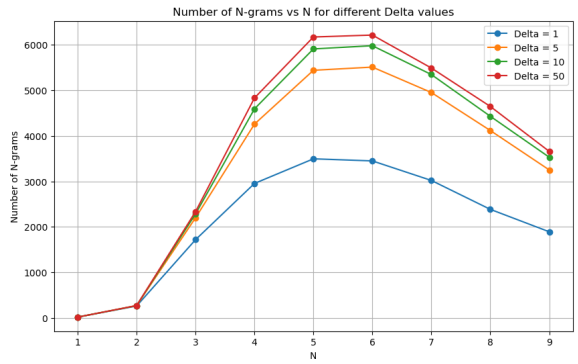

Figure 4. Effect of varying Δ_k on MSNBC dataset

the fundamental nature of the dataset: MSNBC contains user clickstream data, which captures sequences of web page categories or identifiers, not natural language. Since the set of possible navigation paths is broad and less constrained by grammatical or semantic rules, users are more likely to produce a wide variety of unique *n-grams*. Consequently, a larger fraction of the theoretically possible sequences are realized in practice. This contrasts with linguistic datasets, where syntax and context naturally restrict the diversity of plausible *n-grams*, resulting in more saturation and redundancy at lower n values.

2. **Privacy Budget (ϵ)**
The privacy budget ϵ is a central hyper-parameter in differential privacy, directly influencing the amount of noise added to protect individual user data. In the DPNE algorithm, ϵ is used to compute the standard deviations $\sigma_1, \sigma_2, \ldots, \sigma_k$ of the Gaussian noise distributions added at each iteration of histogram construction.

   - **Low ϵ:** A smaller value of ϵ enforces a stronger privacy guarantee. To achieve this, higher noise must be added to the histogram, making it more difficult to distinguish between true and spurious n-gram counts. Consequently, fewer n-grams pass the selection threshold ρ_k, resulting in reduced utility.
   - **High ϵ:** A larger privacy budget corresponds to weaker privacy guarantees but allows for smaller amounts of noise. This improves the accuracy of histogram estimates and increases the number of n-grams that are extracted, thereby enhancing utility.

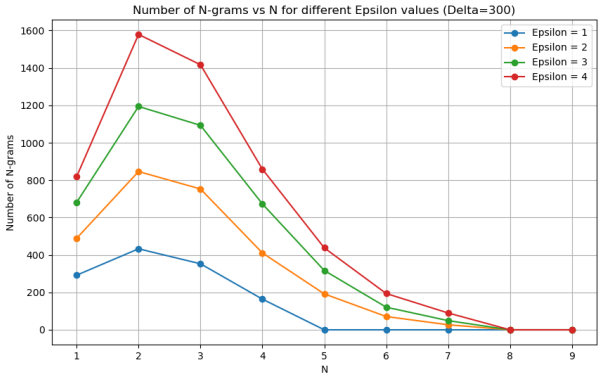

Figure 5. Effect of varying privacy budget ϵ on topical chat dataset

The results of our experiments confirm this expected trade-off between privacy and utility. As shown in Figure 5, increasing ϵ leads to a steady rise in the number of n-grams extracted. This is because lower noise levels make it easier for more n-grams to exceed the selection threshold.

3. **Spurious n-gram Fraction (η)**
The hyper-parameter η controls the fraction of spurious n-grams that are deliberately added to the output to ensure plausible deniability and enhance privacy. For each $k \geq 2$, the number of such spurious k-grams added to the histogram is bounded by $\eta \cdot \min(|S_{k-1}|, |V_k|)$, where S_{k-1} is the

set of surviving $(k-1)$-grams and V_k denotes the vocabulary at the kth level.
In the DPNE algorithm, the number of spurious n-grams B_k is modeled as a random variable:

$$B_k \sim \text{Binomial}(|V_k| - |\text{supp}(H_k)|, \Phi(-\rho_k/\sigma_k)),$$

where Φ is the standard normal CDF, ρ_k is the selection threshold, and σ_k is the standard deviation of the Gaussian noise added in that iteration.

- **Low η:** A small η results in fewer spurious n-grams being added, thereby producing a cleaner output. However, this comes at the cost of reduced plausible deniability and weaker privacy guarantees.
- **High η:** A larger η increases the number of spurious n-grams, making it more difficult to distinguish between genuine and injected outputs. This improves privacy but also introduces more noise into the output, reducing overall accuracy and utility.

The experimental results validate these observations. As depicted in Figure 6, increasing η results in a clear rise in the number of extracted n-grams, most of which are spurious. This demonstrates the trade-off between accuracy and privacy when tuning η.

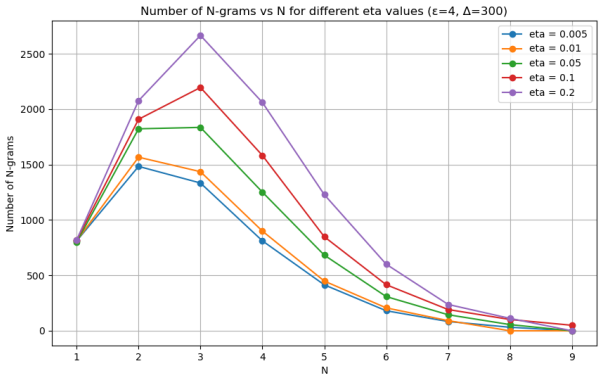

Figure 6. Effect of varying spurious fraction η on the total number of extracted n-grams.

4.4 Verifying the Extracted N-Grams

We now qualitatively examine the extracted n-grams to assess whether the phrases produced by the algorithm are coherent and contextually relevant. The following lists present a few representative examples from different n-gram lengths:

1. **1-grams**: *disney, news, genius, them, social*
2. **2-grams**: *show i, the second, think it, the record, was it*
3. **3-grams**: *i like them, surprised that only, ill have to, i have a, wonder how it*
4. **4-grams**: *you know there was, hi do you watch, i have never been, in a lot of, i never heard of*
5. **5-grams**: *so do you like to, do you like to read, how about you i love, hi do you watch the, but i did not know*
6. **6-grams**: *it was nice talking to you, its been nice chatting with you, i think that would be a, been great chatting with you have, well it was good talking to*
7. **7-grams**: *it was nice chatting with you too, chatting with you have a nice day, great chatting with you too have a, did you know there is a website, you know that the sun is actually*

We observe that as the n-gram length increases, the extracted sequences tend to form complete and natural-sounding conversational phrases. This demonstrates the algorithm's ability to capture longer linguistic patterns while preserving user privacy.

5 Conclusion

In this report, we presented our reimplementation of the scalable DPNE algorithm proposed by Kim et al. for differentially private n-gram extraction. We reproduced the core methodology, conducted experiments on the various datasets, and analyzed the effects of various hyperparameters. Our results are consistent with the findings reported in the original paper.

Acknowledgements

We would like to thank Prof. Sayak Ray Chowdhury and Sarbajit Ghosh for their valuable guidance throughout the course of this project.

Link to Source Code:

References

[1] Kunho Kim et al. *Differentially Private n-gram Extraction*. 2021. arXiv: 2108.02831 [cs.LG]. URL: https://arxiv.org/abs/2108.02831.